Reescrita de Consultas SQL Baseada em Modelos de Linguagem de Grande Escala (LLM-based SQL Query Rewriting)

Luiz Eduardo Treméa[1], **Fahad Kalil**[1]

[1]Ciência da Computação – Atitus Educação
Passo Fundo – RS – Brasil

1123471@atitus.edu.br, fahad.kalil@atitus.edu.br

Abstract. *Escreva seu resumo em língua estrangeira (inglês)...*

Resumo. *Resumo do trabalho (português)...*

1. Introdução

Bancos de dados relacionais estão por trás de boa parte dos sistemas que usamos todos os dias, de aplicações corporativas a plataformas de análise. À medida que esses sistemas crescem e as consultas ficam mais complexas, o tempo que cada consulta leva para ser executada deixa de ser um detalhe e passa a ter impacto direto no custo de infraestrutura e na experiência de quem depende daquela informação. Por isso, otimizar consultas SQL é um problema antigo e, ao mesmo tempo, sempre atual.

Tradicionalmente, esse trabalho fica a cargo do otimizador do próprio Sistema Gerenciador de Banco de Dados (SGBD), que escolhe um plano de execução com base em estimativas de custo e de cardinalidade. Ocorre que essas estimativas nem sempre são precisas, sobretudo quando há correlações complexas entre os dados ou quando a consulta foi escrita de forma pouco eficiente. Nesses casos, reescrever uma consulta mantendo o mesmo resultado, mas mudando como ela é expressa costuma render ganhos que o otimizador sozinho não alcança.

É nesse ponto que os Modelos de Linguagem de Grande Escala (LLMs) ganham relevância. Trabalhos recentes mostram que esses modelos conseguem sugerir reescritas de consultas com bons resultados (Li *et al.*, 2024; Sun *et al.*, 2025), mas também trazem um problema conhecido: a geração livre pode produzir consultas que parecem corretas e não são, seja por alucinação, seja por desvio semântico. Em um ambiente de produção, uma reescrita que retorna dados diferentes do esperado representa um risco inaceitável.

A justificativa deste trabalho parte justamente dessa tensão. Um LLM pode ampliar o espaço de otimização para além das regras fixas, mas só é útil de verdade se houver alguma forma de garantir que a consulta reescrita continua equivalente à original e que ela é, de fato, mais rápida. Em vez de confiar incondicionalmente no modelo, a proposta é deixá-lo sugerir e utilizar o próprio banco de dados como instância de validação.

Disso decorre o problema de pesquisa que orienta o estudo: é possível reescrever consultas SQL com o auxílio de LLMs de modo a melhorar o desempenho sem comprometer a corretude dos resultados?

Para responder a essa pergunta, define-se como objetivo geral desenvolver e avaliar um mecanismo de reescrita de consultas SQL baseado em LLMs, apoiado por etapas de validação sintática, semântica e de desempenho conduzidas pelo próprio SGBD.

Como objetivos específicos, o trabalho se propõe a:

- levantar, na literatura, as principais técnicas de otimização e de reescrita de consultas baseadas em aprendizado de máquina e em modelos de linguagem;
- projetar uma arquitetura não invasiva, capaz de intermediar a comunicação entre a consulta original, o LLM e o banco de dados;
- implementar um processo de validação que combine verificação sintática, comparação semântica de resultados e análise dos planos de execução;
- comparar o desempenho das consultas em três cenários sem reescrita, com reescrita sem validação e com reescrita validada;
- avaliar a viabilidade da abordagem para uso em ambientes reais, considerando os ganhos de desempenho e a preservação da semântica.

2. Referencial Teórico

2.1. Evolução da Otimização de Consultas e Uso de Inteligência Artificial

Durante décadas, a otimização de consultas em bancos de dados se apoiou sobre três pilares: a estimação de cardinalidade, o modelo de custo e a enumeração de planos. A qualidade do plano escolhido depende, em boa parte, de quão bem o otimizador prevê o volume de dados intermediários e de quão coerente é a função de custo em relação aos recursos disponíveis na máquina.

Esse arranjo atende a muitos casos, mas revela limitações importantes em cenários mais exigentes, sobretudo quando existem correlações complexas entre atributos ou quando a carga de trabalho varia com frequência. Nessas situações, um erro de estimação pode levar o otimizador a escolher um plano físico inadequado, com perda direta de desempenho (Marcus; Negi; Mao; Zhang *et al.*, 2019; Marcus; Negi; Mao; Tatbul *et al.*, 2021).

Foi diante desse tipo de limitação que ganhou força o paradigma de *Learned Query Optimization* (LQO), que traz o aprendizado de máquina para dentro do próprio processo de otimização. O *Neo*, por exemplo, emprega redes neurais profundas para aprender padrões de execução a partir de consultas reais e se adaptar ao comportamento dos dados (Marcus; Negi; Mao; Zhang *et al.*, 2019). O *Bao*, por sua vez, adota uma abordagem híbrida baseada em aprendizado por reforço, oferecendo sugestões de execução consulta a consulta e melhorando o desempenho em ambientes reais (Marcus; Negi; Mao; Tatbul *et al.*, 2021).

2.2. Reescrita de Consultas SQL com Modelos de Linguagem

Reescrever uma consulta SQL significa transformá-la em outra que produza exatamente o mesmo resultado, porém de forma mais eficiente. Diferentemente do que ocorre em abordagens de *Text-to-SQL*, aqui o ponto de partida já é uma consulta estruturada; o desafio está em alterá-la sem quebrar a equivalência lógica entre o que entra e o que sai.

O *LLM-R2* recorre a modelos de linguagem para sugerir regras de reescrita e usa aprendizado contrastivo para refinar a forma como as consultas são representadas (Li *et al.*, 2024). O *R-Bot* segue uma linha parecida, mas apoia o modelo na recuperação de evidências estruturais e semânticas, reduzindo os erros ao longo do processo (Sun *et al.*, 2025).

Há ainda propostas que vão além das regras fixas. O *QUITE* organiza múltiplos agentes em um fluxo controlado e testa estratégias de reescrita a partir do retorno do próprio banco de dados (Song *et al.*, 2025). Já o *LITHE* atua diretamente no espaço da consulta (*SQL-to-SQL*): orienta a busca por reescritas a partir das probabilidades de *tokens* do modelo e confirma a equivalência por meio de verificações de correção semântica fundamentadas em análise lógica e estatística (Dharwada *et al.*, 2026).

2.3. Estimativa de Cardinalidade e Modelos Aprendidos

A estimativa de cardinalidade costuma ser apontada como um dos fatores que mais pesam na qualidade do plano de execução, justamente por antecipar o volume de dados que será manipulado em cada etapa da consulta.

O *NeuroCard* ataca esse problema com redes neurais autorregressivas, capazes de capturar correlações complexas entre várias tabelas sem partir do pressuposto de que os atributos são independentes (Yang *et al.*, 2021). O *FLAT* segue outro caminho: combina fatoração independente e condicional em um modelo gráfico, em busca de um meio-termo entre precisão e custo computacional (Zhu *et al.*, 2021).

Em comum, esses trabalhos mostram um avanço claro frente aos métodos tradicionais, com estimativas mais confiáveis e reflexo direto no desempenho das consultas executadas.

2.4. Arquiteturas Baseadas em Recuperação e Agentes

Boa parte das propostas mais recentes combina modelos de linguagem com mecanismos de recuperação de conhecimento, formando pipelines no estilo *Retrieval-Augmented Generation* (RAG). A ideia é converter as consultas em representações vetoriais e, a partir delas, recuperar evidências que ajudem a orientar a geração.

O *R-Bot* ilustra bem essa estratégia ao unir recuperação híbrida e geração guiada por evidências (Sun *et al.*, 2025). O *QUITE* chega a um resultado semelhante por outro meio, distribuindo entre diferentes agentes as tarefas de planejamento, validação e execução das reescritas, em um processo iterativo (Song *et al.*, 2025).

Em conjunto, esses sistemas sugerem que apoiar o modelo em conhecimento externo e em validações sucessivas reduz erros e torna a solução mais robusta (Sun *et al.*, 2025; Song *et al.*, 2025; Dharwada *et al.*, 2026).

2.5. Síntese Crítica da Literatura

A literatura recente aponta para dois caminhos principais. O primeiro leva o aprendizado de máquina para dentro do otimizador, como fazem o *Neo*, o *Bao*, o *NeuroCard* e o *FLAT* (Marcus; Negi; Mao; Zhang *et al.*, 2019; Marcus; Negi; Mao; Tatbul *et al.*, 2021; Yang *et al.*, 2021; Zhu *et al.*, 2021).

O segundo mantém o otimizador intacto e atua de fora, reescrevendo as consultas com o apoio de modelos de linguagem, como é o caso do *LLM-R2*, do *R-Bot*, do *QUITE* e do *LITHE* (Li *et al.*, 2024; Sun *et al.*, 2025; Song *et al.*, 2025; Dharwada *et al.*, 2026).

Independentemente do caminho, os desafios em aberto se repetem: garantir a correção semântica, controlar a latência dos modelos e integrar tudo isso a sistemas que já estão em produção. É nesse encontro entre aprendizado de máquina, reescrita de consultas e recuperação de conhecimento que parece estar o terreno mais promissor para os próximos avanços.

3. Trabalhos Relacionados

Esta seção reúne os principais estudos que aplicam Inteligência Artificial, em especial LLMs e modelos aprendidos, à reescrita e à otimização de consultas SQL. Ao final, destaca-se em que a proposta deste trabalho se diferencia deles.

- **O uso da Inteligência Artificial na otimização de consultas em bancos de dados** (Amaral; Andrade; Campos, 2025)
  Os autores investigam até que ponto o ChatGPT é capaz de reescrever consultas de forma semântica em ambientes PostgreSQL. Para isso, montam um experimento com scripts em Python que automatizam a execução das consultas e a coleta de métricas, com foco em transformações como o uso de *Common Table Expressions* (CTEs), pré-agregações e redução de subconsultas. Em cenários complexos, com muitos *joins*, a reescrita assistida por IA chegou a reduzir o tempo de execução em até 90%, além de racionalizar o uso de índices e tabelas.
- **LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System** (Li *et al.*, 2024)
  O LLM-R2 usa LLMs para sugerir sequências de regras de reescrita voltadas a plataformas como o Apache Calcite. Seu principal trunfo é um modelo contrastivo, treinado por currículo, que seleciona demonstrações (*few-shot*) estruturalmente parecidas com a consulta de entrada. Como a execução final das regras fica a cargo do próprio motor do banco, o sistema garante que a reescrita seja executável e semanticamente equivalente, algo que abordagens de geração livre, mais sujeitas a alucinações, nem sempre conseguem.
- **R-Bot: An LLM-based Query Rewrite System** (Sun *et al.*, 2025)
  O R-Bot busca reduzir alucinações apoiando-se em uma arquitetura de *Retrieval-Augmented Generation* (RAG). A recuperação é híbrida: combina características estruturais e semânticas para localizar "receitas de reescrita" e evidências em fóruns técnicos e manuais de SGBDs. A reescrita acontece passo a passo, com autorreflexão, o que permite ao modelo entender como as regras se relacionam antes de aplicá-las.
- **LITHE: A Query Rewrite Advisor using LLMs** (Dharwada *et al.*, 2026)
  O LITHE trabalha diretamente no espaço da consulta (*SQL-to-SQL*), e não no espaço do plano físico. Ele aplica regras sensíveis ao banco, apoiadas em seletividade e metadados, e as combina com uma busca em árvore de Monte Carlo (*Monte Carlo Tree Search*, MCTS) guiada pelas probabilidades de *tokens* do LLM. Quando a confiança do modelo é baixa, essa estratégia abre espaço para explorar

vários caminhos de reescrita, alcançando ganhos expressivos em relação aos otimizadores nativos.

- **QUITE: A Query Rewrite System Beyond Rules with LLM Agents** (Song *et al.*, 2025)
  O QUITE é um *framework* de múltiplos agentes coordenados por uma Máquina de Estados Finita (*Finite State Machine*, FSM). Ele recebe *feedback* em tempo real do banco por meio de comandos `EXPLAIN` e injeta *hints* para garantir que o plano otimizado seja de fato seguido na execução. Por não depender apenas de regras fixas, consegue encontrar otimizações que transformações pré-definidas não alcançariam.

Diferentemente das abordagens de otimização interna, como o Neo e o Bao, que substituem ou modificam partes do otimizador do SGBD, e dos sistemas presos a regras fixas ou voltados apenas a reduzir latência, este trabalho propõe uma reescrita *SQL-to-SQL* baseada em LLMs, com validação e análise conduzidas pelo próprio gerenciador de banco de dados.

A proposta se diferencia por buscar um equilíbrio entre desempenho e preservação semântica. A geração livre oferece mais liberdade de transformação, mas convive com *semantic drift*, alucinações e regressões de desempenho; os sistemas rígidos, por outro lado, ficam limitados às transformações que alguém previu e codificou de antemão. A ideia aqui é colocar o LLM como agente de reescrita semântica em uma camada externa e não invasiva, enquanto o SGBD assume o papel de validar e fornecer evidências técnicas.

A verificação acontece em etapas. Primeiro, a consulta reescrita passa pelo *parser* do banco, que confere sua validade sintática. Em seguida, comandos como `EXPLAIN` e `EXPLAIN ANALYZE` permitem analisar o plano de execução e comparar o custo estimado entre a versão original e a reescrita. Por fim, comparam-se os resultados das duas consultas para confirmar que a transformação não alterou a semântica original.

O ciclo, então, se fecha assim: o LLM propõe a transformação, o otimizador do SGBD aponta os ganhos de desempenho e os mecanismos de validação asseguram que a reescrita está correta. A meta é chegar a uma solução prática, explicável e compatível com diferentes SGBDs relacionais, confiável o bastante para uso corporativo e ambientes de produção.

4. Materiais e Métodos

Este trabalho se caracteriza como uma pesquisa aplicada, de abordagem quantitativa e natureza experimental. É aplicada porque parte de um problema concreto, a otimização de consultas SQL em bancos de dados; é quantitativa porque recorre a métricas objetivas de desempenho para avaliar os resultados; e é experimental porque envolve a construção de um protótipo funcional e a realização de testes controlados em ambiente computacional.

A proposta é desenvolver um mecanismo de reescrita de consultas SQL baseado em Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs), acompanhado de etapas de validação sintática e semântica e de uma análise de desempenho apoiada no *feedback* do próprio otimizador. Em essência, a pergunta que orienta o estudo é direta: consultas reescritas automaticamente conseguem rodar mais rápido sem comprometer a corretude dos resultados?

4.1. Arquitetura da Solução Proposta

A arquitetura proposta funciona como uma camada intermediária, encarregada de receber as consultas, encaminhá-las ao modelo de linguagem e avaliar se as transformações sugeridas fazem sentido.

O fluxo começa com uma consulta SQL original submetida ao sistema. Ela é então enviada ao LLM, que recebe instruções específicas para devolver uma versão semanticamente equivalente e, idealmente, mais eficiente. Gerada a nova consulta, entram em cena as etapas de validação, responsáveis por verificar tanto a corretude quanto o impacto sobre o desempenho.

Cabe destacar que a solução foi concebida para ser não invasiva: ela não exige nenhuma alteração no núcleo do Sistema Gerenciador de Banco de Dados (SGBD) e pode ser acoplada como complemento aos otimizadores que já existem.

4.2. Processo de Reescrita de Consultas

O processo se inicia com a submissão de uma consulta SQL ao sistema. Essa consulta original serve de entrada para o modelo de linguagem, que recebe um conjunto de instruções voltadas a melhorar a estrutura lógica da consulta.

Entre as transformações esperadas estão a troca de subconsultas por junções, o uso de *Common Table Expressions* (CTEs), a simplificação de predicados, a reorganização de filtros e a eliminação de operações redundantes, estratégias que a literatura já associa à redução de custos de execução e ao melhor aproveitamento dos recursos computacionais.

Cabe reforçar que o modelo não tem autonomia para substituir a consulta original por conta própria. Toda reescrita precisa passar pelas etapas de validação definidas neste trabalho antes de ser aceita como alternativa válida.

4.3. Validação das Consultas Reescritas

Como a literatura registra riscos reais de alucinação e de desvio semântico, a validação foi organizada em múltiplos estágios.

O primeiro é a validação sintática, feita pelo próprio *parser* do banco: consultas com erro de compilação ou incompatibilidades estruturais são descartadas de imediato.

Depois vem a validação semântica. Aqui, a consulta original e a reescrita rodam sobre o mesmo conjunto de dados e seus resultados são comparados; havendo qualquer diferença entre os conjuntos retornados, a reescrita é rejeitada.

Por último, a validação de desempenho se apoia nas informações do otimizador. Os planos gerados por `EXPLAIN` e `EXPLAIN ANALYZE` permitem comparar custos estimados, operadores e tempos de execução. Só é considerada uma otimização válida a consulta que preserva a equivalência semântica e, ao mesmo tempo, melhora o desempenho.

4.4. Ambiente Experimental

Os experimentos serão conduzidos no PostgreSQL, escolhido tanto por sua presença consolidada em ambientes acadêmicos e corporativos quanto pelas ferramentas nativas de análise de desempenho que oferece.

O protótipo será escrito em Python, linguagem que se integra com facilidade a bancos relacionais e a plataformas de Inteligência Artificial. Ainda, entre as bibliotecas Python necessárias, estão: SQLAlchemy, Psycopg, Pandas e Matplotlib.

O SQLAlchemy permite a definição e execução de consultas SQL, enquanto o Psycopg atuará como driver de comunicação com o PostgreSQL. Já o Pandas será usado para manipular e comparar os resultados retornados pelas consultas e, por fim, Matplotlib para gerar os gráficos e apoiar as análises estatísticas.

4.5. Modelo de Linguagem Utilizado

A reescrita propriamente dita ficará a cargo de um Modelo de Linguagem de Grande Escala capaz de lidar com tarefas de *SQL-to-SQL Query Rewriting*.

A depender da disponibilidade de acesso e dos recursos computacionais ao longo do estudo, os experimentos poderão recorrer a modelos como GPT-4o, GPT-5, DeepSeek ou Llama 3. A decisão levará em conta a qualidade das reescritas produzidas, a estabilidade dos resultados e a viabilidade de uso dentro do ambiente experimental montado.

4.6. Conjunto de Dados

A avaliação utilizará bases consagradas na literatura de otimização de consultas e desempenho de bancos de dados. Entre as consideradas estão o TPC-H, o TPC-DS e o *Join Order Benchmark* (JOB), montado a partir da base IMDb. Esses benchmarks reúnem consultas analíticas de diferentes níveis de complexidade, com múltiplas junções, agregações e operações de filtragem. Trabalhar com eles ajuda tanto a comparar os resultados com os de trabalhos relacionados quanto a garantir maior reprodutibilidade dos experimentos.

4.7. Cenários de Avaliação

A avaliação será feita com três cenários distintos. No primeiro, as consultas rodam sem nenhuma alteração, apenas com o otimizador nativo do PostgreSQL. No segundo, passam pela reescrita do modelo de linguagem antes de serem executadas, mas sem nenhum mecanismo adicional de validação. No terceiro, as consultas reescritas percorrem todo o processo de validação proposto aqui, com as verificações sintática, semântica e de plano de execução.

Comparar três cenários permite enxergar não só o potencial de otimização dos LLMs, mas também o quanto os mecanismos de validação ajudam a evitar regressões de desempenho e erros semânticos.

4.8. Métricas de Avaliação

A análise se baseará em métricas objetivas de desempenho: o tempo total de execução, o custo estimado do plano, a quantidade de linhas processadas, a taxa de sucesso das reescritas, o percentual de consultas efetivamente otimizadas e o número de reescritas rejeitadas ao longo da validação.

Esses números serão comparados estatisticamente entre os cenários, o que deve indicar se a abordagem proposta é eficaz e se faz sentido aplicá-la em ambientes reais.

Referências

AMARAL, K. P. N. d.; ANDRADE, L. P. d.; CAMPOS, E. A. V. d. O uso da Inteligência Artificial na otimização de consultas em bancos de dados. **Revista Camalotes – RECAM**, 2025. Disponível em: https://periodicos.insted.edu.br/recam/index. Acesso em: 9 jun. 2026.

DHARWADA, S. *et al.* LITHE: A Query Rewrite Advisor using LLMs. *In:* INTERNATIONAL CONFERENCE ON EXTENDING DATABASE TECHNOLOGY. **Proceedings of the 29th International Conference on Extending Database Technology (EDBT 2026)**. Tampere, Finland: OpenProceedings.org, 2026. Disponível em: https://openproceedings.org/2026/conf/edbt/paper-93.pdf. Acesso em: 9 jun. 2026.

LI, Z. *et al.* LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. **Proceedings of the VLDB Endowment**, v. 18, n. 1, p. 53–65, 2024. Disponível em: https://www.vldb.org/pvldb/vol18/p53-yuan.pdf. Acesso em: 9 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; TATBUL, N. *et al.* Bao: Making Learned Query Optimization Practical. *In:* ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA. **Proceedings of the 2021 International Conference on Management of Data**. Virtual Event, China: ACM, 2021. p. 1275–1288. DOI: 10.1145/3448016.3452838. Disponível em: https://people.csail.mit.edu/hongzi/content/publications/BAO-Sigmod21.pdf. Acesso em: 9 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; ZHANG, C. *et al.* Neo: A Learned Query Optimizer. **Proceedings of the VLDB Endowment**, v. 12, n. 11, p. 1705–1718, 2019. Disponível em: https://www.vldb.org/pvldb/vol12/p1705-marcus.pdf. Acesso em: 9 jun. 2026.

SONG, Y. *et al.* **QUITE: A Query Rewrite System Beyond Rules with LLM Agents**. [*S. l.: s. n.*], 2025. arXiv preprint, abs/2506.07675. DOI: 10.48550/arXiv.2506.07675. Disponível em: https://arxiv.org/abs/2506.07675. Acesso em: 9 jun. 2026.

SUN, Z. *et al.* R-Bot: An LLM-based Query Rewrite System. **Proceedings of the VLDB Endowment**, v. 18, n. 12, p. 5031–5044, 2025. Disponível em: https://www.vldb.org/pvldb/vol18/p5031-li.pdf. Acesso em: 9 jun. 2026.

YANG, Z. *et al.* NeuroCard: One Cardinality Estimator for All Tables. **Proceedings of the VLDB Endowment**, v. 14, n. 1, p. 61–73, 2021. Disponível em: https://www.vldb.org/pvldb/vol14/p61-yang.pdf. Acesso em: 9 jun. 2026.

ZHU, R. *et al.* FLAT: Fast, Lightweight and Accurate Method for Cardinality Estimation. **Proceedings of the VLDB Endowment**, v. 14, n. 9, p. 1489–1502, 2021. Disponível em: https://doi.org/10.14778/3461535.3461539. Acesso em: 9 jun. 2026.